25.3 Canvas 工具

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~190k tokens，输出 ~5k tokens（本节）

前两节分别介绍了 Canvas/A2UI 的概念和 Canvas Host 的实现。这一节分析 Agent 如何通过 Canvas 工具（`src/agents/tools/canvas-tool.ts`）操控 Canvas，以及 Canvas 在不同平台上的表面实现。

25.3.1 push / reset / eval / snapshot 操作

工具注册与 Schema

Canvas 工具定义了 7 个动作（action），统一在一个扁平的 TypeBox Schema 中：

```
// src/agents/tools/canvas-tool.ts

const CANVAS_ACTIONS = [
  "present",     // 显示 Canvas
  "hide",        // 隐藏 Canvas
  "navigate",    // 导航到指定 URL
  "eval",        // 在 Canvas 中执行 JavaScript
  "snapshot",    // 截取 Canvas 快照
  "a2ui_push",   // 推送 A2UI JSONL 数据
  "a2ui_reset",  // 重置 A2UI 渲染状态
] as const;

const CanvasToolSchema = Type.Object({
  action: stringEnum(CANVAS_ACTIONS),       // 必填：动作类型
  node: Type.Optional(Type.String()),       // 目标节点
  // present 专用
  target: Type.Optional(Type.String()),     // URL 或路径
  x: Type.Optional(Type.Number()),          // 放置坐标
  y: Type.Optional(Type.Number()),
  width: Type.Optional(Type.Number()),      // 放置尺寸
  height: Type.Optional(Type.Number()),
  // navigate 专用
  url: Type.Optional(Type.String()),
  // eval 专用
  javaScript: Type.Optional(Type.String()),
  // snapshot 专用
  outputFormat: optionalStringEnum(["png", "jpg", "jpeg"]),
  maxWidth: Type.Optional(Type.Number()),
  quality: Type.Optional(Type.Number()),
  delayMs: Type.Optional(Type.Number()),
  // a2ui_push 专用
  jsonl: Type.Optional(Type.String()),      // 内联 JSONL
  jsonlPath: Type.Optional(Type.String()),  // JSONL 文件路径
});
```

这是一个**扁平化设计**——所有动作的参数混在一个 Schema 中，运行时根据 `action` 字段验证必要参数。这样做的好处是 LLM 只需要学习一个工具的参数结构，而非 7 个独立工具。

Gateway 代理模式

Canvas 工具的所有操作都不直接与 Canvas Host 通信，而是通过 Gateway 的 `node.invoke` 进行转发：

```typescript
// src/agents/tools/canvas-tool.ts — invoke 辅助函数

const nodeId = await resolveNodeId(gatewayOpts, params.node, true);

const invoke = async (command: string, invokeParams?: Record<string, unknown>) =>
  await callGatewayTool("node.invoke", gatewayOpts, {
    nodeId,                            // 目标节点 ID
    command,                           // 如 "canvas.present"
    params: invokeParams,              // 命令参数
    idempotencyKey: crypto.randomUUID(), // 幂等键
  });
```

数据流：

```
Agent 调用 canvas 工具
    │
    ↓ callGatewayTool("node.invoke", ...)
Gateway 协议
    │
    ↓ node.invoke RPC
原生应用节点（iOS/Android/macOS）
    │
    ↓ 节点命令执行器
WebView 中执行对应操作
```

每次调用都附带一个 `idempotencyKey` （幂等键），确保网络重传不会导致操作被重复执行。

逐个动作解析

present：显示 Canvas

```
case "present": {
  const placement = {
    x: typeof params.x === "number" ? params.x : undefined,
    y: typeof params.y === "number" ? params.y : undefined,
    width: typeof params.width === "number" ? params.width : undefined,
    height: typeof params.height === "number" ? params.height : undefined,
  };
  const invokeParams: Record<string, unknown> = {};
  if (typeof params.target === "string" && params.target.trim()) {
    invokeParams.url = params.target.trim();
  }
  if (Number.isFinite(placement.x) || ...) {
    invokeParams.placement = placement;
  }
  await invoke("canvas.present", invokeParams);
  return jsonResult({ ok: true });
}
```

`present` 可以指定：

- **target**——要加载的 URL（通常指向 Canvas Host 的路径，如 `http://localhost:PORT/__openclaw__/canvas/`）
- **placement**——Canvas 窗口在屏幕上的位置和大小（`x` / `y` / `width` / `height`），支持精确定位

hide：隐藏 Canvas

```
case "hide":
  await invoke("canvas.hide", undefined);
  return jsonResult({ ok: true });
```

最简单的操作——无参数，直接隐藏。

navigate：导航到 URL

```
case "navigate": {
  const url = readStringParam(params, "url", { required: true });
  await invoke("canvas.navigate", { url });
  return jsonResult({ ok: true });
}
```

在已显示的 Canvas WebView 中导航到新的 URL，不需要先 hide 再 present。

eval：执行 JavaScript

```
case "eval": {
  const javaScript = readStringParam(params, "javaScript", { required: true });
  const raw = await invoke("canvas.eval", { javaScript });
  const result = raw?.payload?.result;
  if (result) {
    return { content: [{ type: "text", text: result }], details: { result } };
  }
  return jsonResult({ ok: true });
}
```

`eval` 在 Canvas WebView 中执行任意 JavaScript 代码，并返回执行结果。这是 Canvas 最灵活的操作——Agent 可以用它来动态修改页面、读取 DOM 状态、触发事件等。返回值如果存在，会作为文本内容返回给 Agent。

snapshot：截取快照

```
case "snapshot": {
  const format = formatRaw === "jpg" || formatRaw === "jpeg" ? "jpeg" : "png";
  const raw = await invoke("canvas.snapshot", { format, maxWidth, quality });

  // 解析快照负载
  const payload = parseCanvasSnapshotPayload(raw?.payload);

  // 保存为临时文件
  const filePath = canvasSnapshotTempPath({
    ext: payload.format === "jpeg" ? "jpg" : payload.format,
  });
  await writeBase64ToFile(filePath, payload.base64);

  // 返回图片结果（base64 + 文件路径）
  return await imageResult({
    label: "canvas:snapshot",
    path: filePath,
    base64: payload.base64,
    mimeType: imageMimeFromFormat(payload.format) ?? "image/png",
  });
}
```

`snapshot` 是 Canvas 工具中最复杂的操作：

1. 发送 `canvas.snapshot` 命令到节点，节点在 WebView 中截图
2. 节点返回 base64 编码的图片数据
3. `parseCanvasSnapshotPayload()` 解析负载（验证 `format` 和 `base64` 字段）
4. `writeBase64ToFile()` 将 base64 写入临时文件（`/tmp/openclaw-canvas-snapshot-{uuid}.{ext}`）
5. 以 `imageResult` 格式返回，Agent 可以"看到"截图内容

快照临时文件的路径格式：

```
// src/cli/nodes-canvas.ts
export function canvasSnapshotTempPath(opts: { ext: string }) {
  const tmpDir = opts.tmpDir ?? os.tmpdir();
  const id = opts.id ?? randomUUID();
  const ext = opts.ext.startsWith(".") ? opts.ext : `.${opts.ext}`;
  return path.join(tmpDir, `${cliName}-canvas-snapshot-${id}${ext}`);
  // 例: /tmp/openclaw-canvas-snapshot-a1b2c3d4.jpg
}
```

a2ui_push：推送 A2UI 数据

```
case "a2ui_push": {
  // 支持内联 JSONL 或文件路径两种方式
  const jsonl =
    typeof params.jsonl === "string" && params.jsonl.trim()
      ? params.jsonl                                          // 内联
      : typeof params.jsonlPath === "string"
        ? await fs.readFile(params.jsonlPath.trim(), "utf8")  // 从文件读取
        : "";
  if (!jsonl.trim()) {
    throw new Error("jsonl or jsonlPath required");
  }
  await invoke("canvas.a2ui.pushJSONL", { jsonl });
  return jsonResult({ ok: true });
}
```

a2ui_push 支持两种输入方式：

- **jsonl** ——直接在参数中提供 JSONL 字符串（适用于少量 UI 指令）
- **jsonlPath** ——指定 JSONL 文件路径（适用于大量 UI 指令，Agent 先写文件再推送）

a2ui_reset：重置 A2UI 状态

```
case "a2ui_reset":
  await invoke("canvas.a2ui.reset", undefined);
  return jsonResult({ ok: true });
```

清除 A2UI 渲染器的所有状态，恢复到初始空白状态。

七个动作总结

动作	Gateway 命令	必要参数	返回
`present`	`canvas.present`	无（可选 target、placement）	`{ ok: true }`
`hide`	`canvas.hide`	无	`{ ok: true }`
`navigate`	`canvas.navigate`	`url`	`{ ok: true }`
`eval`	`canvas.eval`	`javaScript`	文本结果或 `{ ok: true }`
`snapshot`	`canvas.snapshot`	无（可选 format、maxWidth）	图片（base64 + 文件）
`a2ui_push`	`canvas.a2ui.pushJSONL`	`jsonl` 或 `jsonlPath`	`{ ok: true }`
`a2ui_reset`	`canvas.a2ui.reset`	无	`{ ok: true }`

25.3.2 Canvas 在 macOS / iOS / Android 上的表面

节点命令策略

Canvas 命令在不同平台上的可用性由 `node-command-policy.ts` 中的**节点命令策略**控制：

```
// src/gateway/node-command-policy.ts

const CANVAS_COMMANDS = [
  "canvas.present", "canvas.hide", "canvas.navigate",
  "canvas.eval", "canvas.snapshot",
  "canvas.a2ui.push", "canvas.a2ui.pushJSONL", "canvas.a2ui.reset",
];

const PLATFORM_DEFAULTS: Record<string, string[]> = {
  ios:     [...CANVAS_COMMANDS, ...CAMERA, ...SCREEN, ...LOCATION],
  android: [...CANVAS_COMMANDS, ...CAMERA, ...SCREEN, ...LOCATION, ...SMS],
  macos:   [...CANVAS_COMMANDS, ...CAMERA, ...SCREEN, ...LOCATION, ...SYSTEM],
  linux:   [...SYSTEM_COMMANDS],       // 无 Canvas
  windows: [...SYSTEM_COMMANDS],       // 无 Canvas
};
```

Canvas 在 **iOS**、**Android**、**macOS** 三个平台上可用，在 **Linux 和 Windows** 上不可用。这是因为 Canvas 依赖原生应用的 WebView——Linux 和 Windows 上的 OpenClaw 以无头（headless）模式运行，没有图形界面。

命令访问控制

节点命令策略实现了三重访问控制：

```typescript
// src/gateway/node-command-policy.ts

export function resolveNodeCommandAllowlist(cfg, node): Set<string> {
  // 1. 平台默认允许列表
  const platformId = normalizePlatformId(node?.platform, node?.deviceFamily);
  const base = PLATFORM_DEFAULTS[platformId] ?? PLATFORM_DEFAULTS.unknown;

  // 2. 配置文件扩展允许
  const extra = cfg.gateway?.nodes?.allowCommands ?? [];

  // 3. 配置文件禁止
  const deny = new Set(cfg.gateway?.nodes?.denyCommands ?? []);

  // 合并：(平台默认 ∪ 配置扩展) − 配置禁止
  const allow = new Set([...base, ...extra]);
  for (const blocked of deny) allow.delete(blocked);
  return allow;
}

export function isNodeCommandAllowed(params): { ok: true } | { ok: false; reason: strin
  // 检查 1：命令在允许列表中
  if (!params.allowlist.has(command)) {
    return { ok: false, reason: "command not allowlisted" };
  }
  // 检查 2：节点声明了该命令
  if (!params.declaredCommands?.includes(command)) {
    return { ok: false, reason: "command not declared by node" };
  }
  return { ok: true };
}
```

检查层级	说明	示例
平台允许列表	基于节点平台的默认命令	iOS 默认允许 Canvas + Camera
配置扩展	`gateway.nodes.allowCommands` 可添加额外命令	允许 Linux 节点使用 Canvas
配置禁止	`gateway.nodes.denyCommands` 可屏蔽命令	禁止 Android 节点截图
节点声明	节点注册时声明自己支持的命令	节点未声明 Canvas 则不可用

各平台的 WebView 实现

Canvas 工具通过 Gateway 向节点发送命令，但**实际的 WebView 渲染发生在原生应用侧**。不同平台的实现：

平台	WebView 技术	Canvas 实现位置	特性
macOS	`WKWebView`	`apps/macos/` (Swift)	桌面窗口，支持 placement 定位，支持 system.run
iOS	`WKWebView`	`apps/ios/` (Swift)	全屏或分屏，安全区域适配，支持 camera/location
Android	`android.webkit.WebView`	`apps/android/` (Kotlin)	全屏或分屏，高对比度主题，支持 SMS

三个平台的 WebView 都需要：

1. **加载 Canvas Host URL**——通常是 `http://<gateway-host>:<canvas-port>/__openclaw__/a2ui`
2. **注入 JS Bridge**——iOS 通过 `WKScriptMessageHandler`，Android 通过 `@JavascriptInterface`
3. **处理节点命令**——接收 `canvas.present` / `canvas.eval` 等命令并在 WebView 中执行

CLI 命令行接口

除了 Agent 工具，Canvas 操作也可以通过 CLI 直接调用：

```bash
# 截取快照
openclaw nodes canvas snapshot --node my-iphone --format jpg --max-width 1024

# 显示 Canvas
openclaw nodes canvas present --node my-iphone --target http://localhost:8080/

# 在 Canvas 中执行 JS
openclaw nodes canvas eval --node my-iphone --js "document.title"

# 推送 A2UI 文本
openclaw nodes canvas a2ui push --node my-iphone --text "Hello from CLI"

# 推送 A2UI JSONL 文件
openclaw nodes canvas a2ui push --node my-iphone --jsonl ./ui-commands.jsonl

# 重置 A2UI
openclaw nodes canvas a2ui reset --node my-iphone
```

CLI 实现在 `src/cli/nodes-cli/register.canvas.ts` 中，它与 Agent 工具共享相同的 `invokeCanvas()` 辅助函数，最终都通过 `callGatewayCli("node.invoke", ...)` 调用 Gateway。

CLI 的 `a2ui push` 命令还额外支持 `--text` 参数——快速将纯文本转换为 A2UI JSONL 格式，无需手动编写 JSONL：

```typescr
// src/cli/nodes-cli/register.canvas.ts — a2ui push

const jsonl = hasText
  ? buildA2UITextJsonl(String(opts.text))  // 纯文本 → JSONL
  : await fs.readFile(String(opts.jsonl), "utf8");

const { version, messageCount } = validateA2UIJsonl(jsonl);
if (version === "v0.9") {
  throw new Error(
    "Detected A2UI v0.9 JSONL (createSurface). OpenClaw currently supports v0.8 only."
  );
}
```

注意版本检查：当前 OpenClaw 只支持 A2UI v0.8 协议，如果检测到 v0.9 的 `createSurface` 指令会报错。

端到端工作流示例

一个完整的 Canvas 使用场景——Agent 为用户生成数据可视化：

```
1. Agent 生成 HTML 文件
   → 写入 ~/.openclaw/canvas/chart.html

2. Agent 调用 canvas present
   → canvas-tool.ts → callGatewayTool("node.invoke")
   → Gateway → node.invoke → iOS 应用
   → iOS 打开 WebView，加载 http://host:port/__openclaw__/canvas/chart.html
   → Canvas Host 提供 chart.html（注入 Live Reload）
   → WebView 渲染图表

3. Agent 调用 canvas snapshot
   → 同样路径到达 iOS 应用
   → iOS 截取 WebView 快照
   → Base64 编码回传
   → Agent "看到" 渲染结果，确认正确

4. 用户与图表交互（点击某个数据点）
   → WebView 中的 JS 调用 openclawSendUserAction()
   → iOS WKScriptMessageHandler 接收
   → 消息回传到 Agent
   → Agent 根据用户操作更新图表
```

本节小结

1. Canvas 工具将 7 个动作统一为一个**扁平化 Schema**，LLM 只需学习一个工具接口
2. 所有操作都通过 `callGatewayTool("node.invoke")` **代理到目标节点**执行，不直接与 Canvas Host 通信
3. `snapshot` 是最复杂的操作，涉及节点截图 → base64 编码 → 解析负载 → 写入临时文件 → 返回图片结果
4. `a2ui_push` 支持内联 JSONL 和文件路径两种输入方式，CLI 额外支持 `--text` 快捷方式
5. Canvas 在 **iOS / Android / macOS** 可用，Linux / Windows 因缺乏 WebView 而不支持
6. **节点命令策略**实现四重访问控制：平台默认 → 配置扩展 → 配置禁止 → 节点声明
7. CLI 和 Agent 工具共享相同的底层调用链，都通过 Gateway 的 `node.invoke` 到达目标节点